

Cluster Then Classify — what the project does and how

1. The problem

Supervised machine learning needs labelled data. Someone has to read every document and tag it. For 7,600 news articles at a realistic 200 documents an hour, that is about 38 hours of tedious human work before any model can be trained at all.

This is the bottleneck for most real text-classification projects. The model is the easy part. The labels are the expensive part.

The question this project asks: can an LLM produce those labels instead, and if so, how much accuracy do you lose by not paying a human?

Notice that the question has a number as its answer. That is deliberate. "Can an LLM label data?" is a yes/no question anyone can hand-wave. "How much worse is it than a human?" requires an experiment.

2. The core idea

The naive approach is to send all 7,600 articles to an LLM one at a time and ask it to classify each one. That works, but it is 7,600 API calls, it costs real money, and it means you need the LLM forever — every future document has to go through the API too.

This project does something cheaper:

1. **Group the articles automatically** by similarity. No labels needed, no AI needed — this is standard clustering.
2. **Ask the LLM to name each group.** Four groups means four names, which is **one** API call, not 7,600.
3. **Every article inherits its group's name.** Now all 7,591 articles are labelled.
4. **Train an ordinary classifier** on those labels. Once trained, it runs on its own — no API, no cost, milliseconds per document.
5. **Check the answer** against the real labels, which were hidden at the start.

The clustering does the heavy lifting of sorting the documents. The LLM does the one thing clustering cannot do: attach human-meaningful names to the groups it found.

3. The dataset, and the trick that makes the project rigorous

AG News: 7,600 news articles, four categories — World, Sports, Business, Sci/Tech — with exactly 1,900 articles each.

It already has correct labels. That looks like it defeats the purpose, but it is precisely why it was chosen.

Phase 0 hides the labels in a separate file and does not open it again until Phase 5.

The pipeline runs completely blind. Every decision — how many clusters, which algorithm, what to name the categories — gets made without ever seeing the answers. Only at the very end are the true labels revealed and the results scored.

This is the difference between a demonstration and an experiment. Without the quarantine, you could unconsciously tune choices until the output matched what you already knew, and the final accuracy would mean nothing. With it, the number is earned.

It also removes the need to hand-label a validation set, which would otherwise cost a day of work and give you a few hundred rows instead of 7,591.

4. How each phase works

Phase 0 — Cleaning and quarantine

Each row is a headline and a description. They get joined into a single text.

The cleaning is not boilerplate — every rule was added because it visibly changed the result:

Source markers. Most descriptions begin with the news agency: (AP -), (Reuters -), (LOS ANGELES (Reuters) -). Left in, the strongest signal in the entire corpus is *which agency filed the story*. Clusters form around Reuters vs AP instead of around sports vs business.

Title and description cleaned separately, then joined. The prefix patterns anchor to the start of a field. Once joined, the description no longer starts the string, so the patterns silently never fire. This bug produced no error and no warning — the text simply stayed dirty.

Broken HTML entities. Many rows contain (#39;) and (quot;) — missing the leading (&). Python's (html.unescape) passes straight over them. They have to be repaired first, or the word (quot) becomes a common token.

Hidden markup. Once entities are unescaped, real HTML appears. Business stories embed Reuters stock-quote links: (). Left in, the Business cluster partly forms around the words (fullquote), (aspx) and (reuters) — around markup, not around business.

Fixing these moved cluster accuracy from **61.3% to 65.6%**. The cleaning was worth more than any model tuning that followed.

Then 9 rows are dropped as duplicates or too short, leaving **7,591**. The labels are written to a separate file and removed from memory.

Phase 1 — Embeddings

A computer cannot compare sentences directly. Each article is converted into a list of 384 numbers using MiniLM, a sentence-embedding model.

The useful property: articles with similar meanings get similar numbers. "Dolphins beat Rams" and "Yankees win in extra innings" land close together. "Microsoft releases

software" lands far away.

Think of it as placing every article at a position on a map, where distance means dissimilarity. Nothing has been labelled — the model just learned during its own training that certain words pattern together.

These get cached to disk, because Phase 2 gets re-run many times and recomputing them each pass wastes an hour a day.

Phase 2 — Clustering

Look at that map and find the dense clumps. KMeans draws boundaries around them.

The computer now knows *these 1,942 articles belong together* but has no idea what they are. They are Group 0, Group 1, Group 2, Group 3.

Choosing the number of groups is the risky decision, and there is a trap here worth knowing about. The standard metric is the silhouette score. On this data it rose steadily as the number of clusters increased — 0.035 at $k=4$, 0.061 at $k=15$ — while actual agreement with the true categories *fell by more than half*. Choosing k by silhouette would have given the worst available answer.

Silhouette measures whether clusters are geometrically tight. It cannot measure whether they are *meaningful*. So the pipeline records it for the writeup and ignores it for the decision.

The decision is made by reading. Pull the 10 articles closest to each cluster's centre and read them. If you can say in one sentence what a cluster is about, it is real. If it reads as assorted news, it is not, and everything downstream inherits that failure.

Phase 3 — LLM labelling

Take the 15 articles nearest each cluster's centre and show them to Gemini: *here are four groups of documents, give each one a category name*.

Four decisions here matter more than the code:

The prompt never says what the dataset is. It does not mention news, or four categories. Telling the model would hand it the answer and make Phase 5 meaningless. It has to work out the categories from the documents alone.

All four clusters go in one call, not four. Labelled separately, two clusters both come back "Technology News" — neither call knew the other existed. Together, the model is forced to make them mutually exclusive, which is what a classification target requires.

Centroid-nearest documents, not a random sample. Documents near the centre are the cluster's core. A random draw pulls in boundary cases that make coherent clusters look muddled and push the model toward vague labels like "General News".

The model can refuse. It returns a `coherent` flag and a confidence score, and is explicitly told not to invent a label to be agreeable. A cluster marked incoherent is a signal to go back to Phase 2, not to press on.

Then **propagation**: every article inherits its cluster's name. 7,591 labelled articles, from one API call, with zero human annotation.

Propagation is also where error enters. An article at the edge of the Sports cluster gets labelled Sports even if it is really about a stadium financing deal. The pipeline estimates this without ground truth by asking the model to re-label 300 individual documents and measuring how often that disagrees with the propagated label.

Phase 4 — The classifier

Train ordinary logistic regression on TF-IDF features, using the inherited labels. Nothing exotic — the point is that the labels are free, not that the model is clever.

This phase produces the most misleading number in the project. It scored 0.953 on its held-out split. That looks excellent and means almost nothing: the held-out split carries *inherited* labels too, so the score measures how faithfully the classifier reproduced the clustering — not whether the clustering was right.

A model can reproduce a wrong grouping perfectly. Reporting 95% here would be the single biggest mistake available in this project.

Phase 5 — The reveal

Open the true labels.

Align the clusters. Cluster names are arbitrary; "Group 2" has to be matched to a real category. This uses the Hungarian algorithm on the contingency table, which finds the pairing that maximises agreement.

Two rules: don't eyeball it, because you will pick the pairing that flatters you. And never match on name similarity — if the LLM output "Sports" and you matched it to the true class "Sports" by string comparison, you would be rewarding it for guessing which public dataset this is rather than for understanding the documents.

Run the ablations. Five models, each trained on a different label source, all scored against the same true test labels.

5. The results

From the offline run (weaker stand-in embeddings — real MiniLM should be better):

Trained on	Accuracy	% of ceiling
100% true labels (ceiling)	0.884	100%
LLM labels, zero human effort	0.728	82.4%
100 true labels + LLM for the rest	0.730	82.6%
100 true labels only	0.600	67.8%
Shuffled labels (floor)	0.249	28.1%

The headline: 82.4% of fully supervised performance, with nobody labelling anything.

The row that proves it was worth doing: hand-labelling 100 articles gets 67.8%. Bootstrapping beats it comfortably. If it had lost to that row, the pipeline would not be worth building — and you would have to say so.

The row that reveals a limitation: adding 100 true labels to the bootstrapped set moved the result by 0.2 percentage points. Nothing. That says the bottleneck is not how many labels you have — it is that the cluster boundaries themselves are wrong in places. More human labelling will not fix it; better separation would.

Cost: about 3 cents in API calls against an estimated \$949 of human annotation time. Roughly 30,000× cheaper.

A finding worth reporting: the number of clusters does not have to match the number of categories. At k=8, accuracy rose to 0.807 — **91.8% of ceiling**, much better than k=4. Surplus clusters map many-to-one onto categories, letting a broad category split into the sub-topics that actually cluster.

6. Where it fails, and why that is fine

Business and Sci/Tech do not separate. One cluster holds 1,287 Sci/Tech articles alongside 400 Business ones, and Business itself splits across two clusters.

This is not a bug and it survived every cleaning fix. A story about a chip manufacturer's quarterly earnings is honestly both business and technology. The boundary is genuinely blurred in the world, so it is blurred in the embedding space too.

Naming this clearly, and explaining *why* the classes are hard to separate, is what makes the rest of the report credible. A project that reports only successes reads as unexamined.

7. What to say when someone asks you to defend it

"Why not just ask the LLM to classify every document?" 7,600 API calls instead of 1, and you would need the LLM permanently. This trains a standalone classifier that runs for free forever.

"How do you know the labels are any good?" Two ways. The spot check, which measures propagation noise without ground truth. And Phase 5, which measures everything against real labels that were hidden throughout.

"Isn't 82% just a number you picked?" It is a ratio against a ceiling measured on the same test set, with a random floor and a small-supervised baseline for comparison. All five conditions are in the ablation table.

"Couldn't you have tuned things until it worked?" The labels were in a sealed file from Phase 0 to Phase 5. Every choice was made blind.

"Why is the classifier's own score 95% but you report 73%?" Because 95% measures agreement with the clustering, and the clustering is not ground truth. Being able to explain that distinction is most of what this project demonstrates.

8. Honest limitations

- The reported numbers use TF-IDF stand-in embeddings, not MiniLM. Real embeddings should improve them; they have not been run yet.
- 7,600 rows is the AG News *test* split. The 120k train split would give more data and probably cleaner clusters.
- Cost figures are order-of-magnitude estimates, not measurements.
- Everything downstream of the API response is tested; the live API call itself has not been run.
- Results depend on the corpus having genuine topical structure. On a dataset where the target is subtle — sentiment, for instance — clustering follows topic instead and the whole approach degrades.